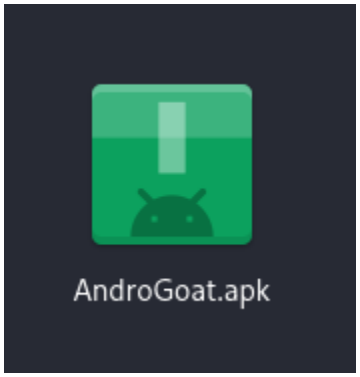
Assignment 1

Objective:

To analyze the APK file and identify the OWASP-listed security risk using apktool, jadx, and bytecode viewer.

Apk used:



Analyzing the file using - Apktool

```
┌──(kali㉿kali)-[~/Downloads]
└─$ apktool d AndroGoat.apk

I: Using Apktool 2.7.0-dirty on AndroGoat.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/kali/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...

┌──(kali㉿kali)-[~/Downloads]
└─$
```

Submitted by Nanda Krishnan V *, MSc Cybersecurity*

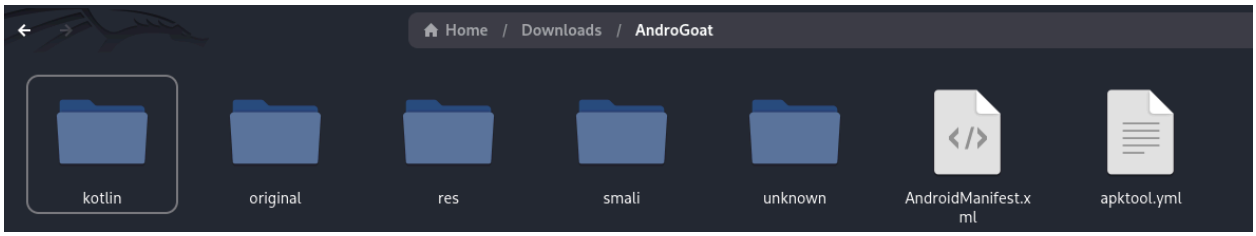
Command: apktool d AndroGoat.apk

- apktool: A tool to decode Android apps.
- d: The "decode" command.

Action: The tool is "unpacking" the .apk file.

- It decodes the AndroidManifest.xml (which defines app permissions and components).
- It converts the app's compiled code (classes.dex) into human-readable .smali files ("Baksmaling").
- It copies all other assets, like images and libraries.

After the execution, we received a new folder containing the decompressed files. The folder consists of these files:



We can see the AndroidManifest.xml file is the app's "instruction manual" for the Android operating system. It declares the app's identity, what permissions it needs (like INTERNET or CAMERA), and lists all its components (like screens and services) so the system knows how to run it.

After the manual scanning, we concluded that some vulnerabilities can be matched with OWASP top 10 Mobile, they are as follows:

M1: Improper Platform Usage

- The activity is named AccessControl1ViewActivity, which implies it's a screen that should be protected. Normally, a user would have to log in or navigate through the app to reach this screen.
- The <intent-filter> you see creates a "deep link" (androgoat://vulnapp). Any other app on the phone (like a web browser or a malicious app) can use this link to launch that screen *directly*, bypassing any login screens or other security controls.

```
    <category android:name="android.intent.category.DEFAULT"/>
    <data android:host="vulnapp" android:scheme="androgoat"/>
  </intent-filter>
</activity>
<receiver android:enabled="true" android:exported="true" android:name="owasp.sat.agoat.ShowDataReceiver"/>
<activity android:label="@string/hardcode" android:name="owasp.sat.agoat.HardCodeActivity"/>
<activity android:label="@string/sql" android:name="owasp.sat.agoat.InsecureStorageSQLiteActivity"/>
<activity android:label="@string/sp2" android:name="owasp.sat.agoat.InsecureStorageSharedPrefs1Activity"/>
<activity android:label="@string/network" android:name="owasp.sat.agoat.TrafficActivity"/>
<activity android:name="owasp.sat.agoat.ContentProviderActivity"/>
<activity android:label="@string/emulator" android:name="owasp.sat.agoat.EmulatorDetectionActivity"/>
```

- The activity is named HardCodeActivity, which implies that its corresponding code file (HardCodeActivity.java) contains **hardcoded secrets**.
- This means a developer has insecurely written sensitive data (like an API key, a password, or an encryption key) directly into the application's source code.
- An attacker can use any decompiler (like jadx or apktool) to reverse-engineer the app. They can then simply open the HardCodeActivity source code and read the secret in plain text, giving them full access to whatever that key protects.

M5: Insecure Communication

```
<manifest package="owasp.sat.agoat">
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
+<application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:networkSecurityConfig="@xml/
network_security_config" android:roundIcon="@mipmap/ic_launcher_round" android:supportsRtl="true" android:theme="@style/AppTheme">
  </application>
</manifest>
```

This attribute tells the app **not** to use the default system rules for network connections. Instead, it points to a custom file (named network_security_config.xml) that defines the app's own security policies for web traffic.

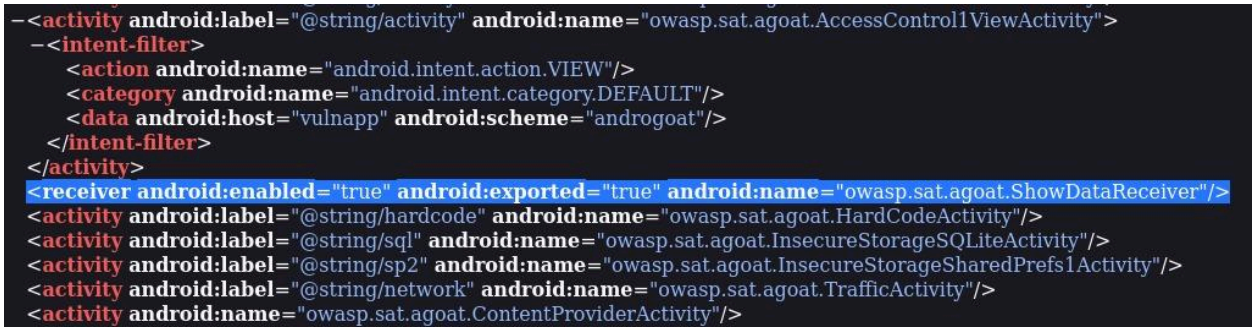
This file is critical because it controls:

1. **Cleartext Traffic (HTTP):** Whether the app is allowed to send unencrypted http:// traffic to specific domains.
2. **Certificate Pinning:** Whether the app will "pin" and trust *only* a specific SSL certificate for its servers, which is a strong defense against man-in-the-middle attacks.
3. **Custom Trust Anchors:** Whether the app should trust custom or user-added certificates (like those used for corporate proxies or by attackers).

M8: Security Misconfiguration.

- **android:allowBackup="true"**: This allows anyone with physical access to the device to use a developer tool (adb) to copy all of the app's private data (like user preferences, databases, and login tokens).

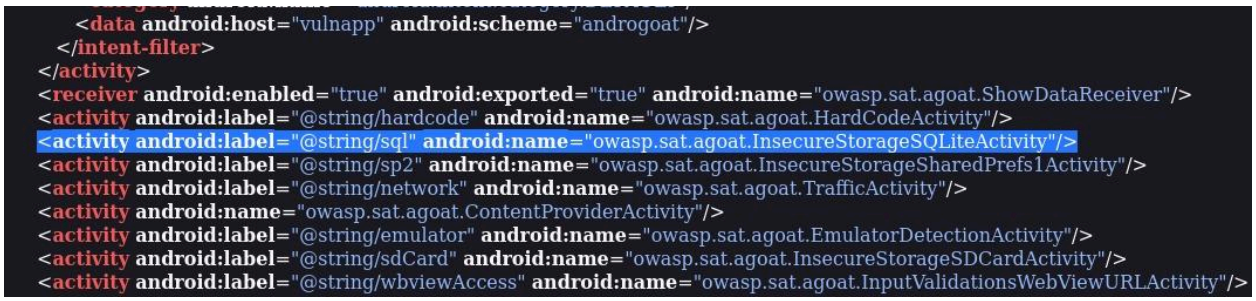
- The activity is named BinaryPatchingActivity, which implies its code contains a **weak security check** (for example, a simple "if" statement that checks if the user is a premium member).
- This screen is intentionally vulnerable to "binary patching," where an attacker uses reverse engineering tools (like apktool and smali) to find and modify the app's compiled code. An attacker can alter that "if" statement always to return true, effectively unlocking the premium feature for free. They then recompile and reinstall the modified app.



- The component is a BroadcastReceiver named ShowDataReceiver, which implies it's a listener designed to receive messages (broadcasts) and "show data" when triggered.

- The vulnerability is the android:exported="true" flag. This setting allows any other app on the device, including a malicious one, to send a broadcast message to this receiver.
- An attacker can craft a malicious "intent" (message) and send it directly to this receiver to execute its code. Since it's an internal component, it likely doesn't check who sent the message.

M9: Insecure Data Storage



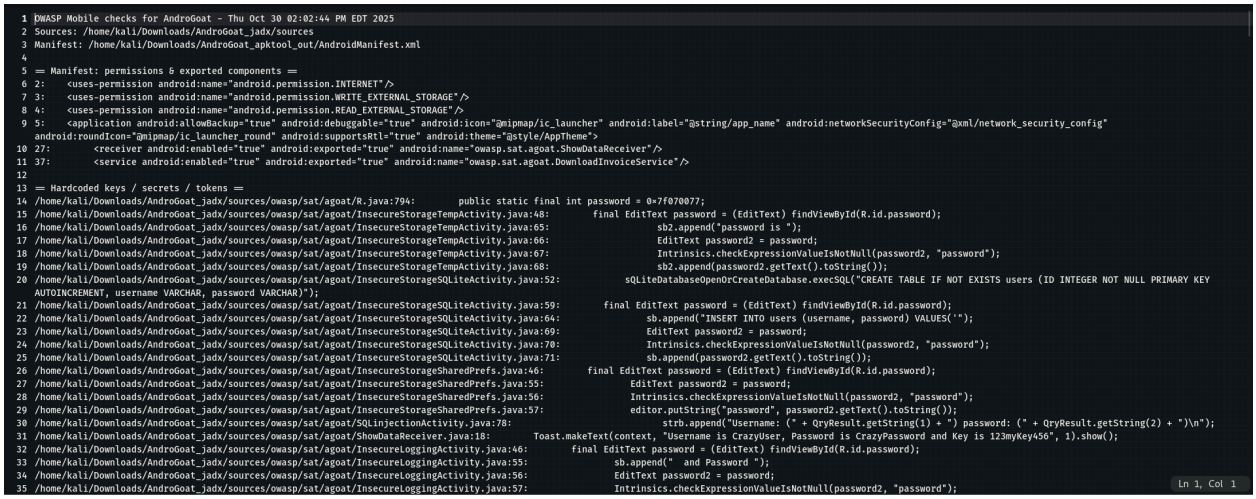
- The activity is named InsecureStorageSQLiteActivity, which implies it's a screen that saves data to a SQLite database.
- The "Insecure" part of the name indicates that the app's code writes this data (like user credentials or personal notes) to the database file in **plain, unencrypted text**.

An attacker with access to the device (either through root or by exploiting a flaw like allowBackup="true") can steal this database file. They can then open it with any standard database viewer and read all the sensitive information inside, completely bypassing the app.

Analyzing APK - Jadx

In this tool i ran the command to analyze the full apk and make the analysis report to *AndroGoat_owasp_checks.txt*

After the execution, we got the text file and next i analyzed the file for any vulnerabilities



```
OWASP Mobile checks for AndroGoat - Thu Oct 30 02:02:44 PM EDT 2025
Sources: /home/kali/Downloads/AndroGoat_jadx/sources
Manifest: /home/kali/Downloads/AndroGoat_apktool_out/AndroidManifest.xml

== Manifest: permissions & exported components ==
2:    <uses-permission android:name="android.permission.INTERNET"/>
3:    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
4:    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
5:    <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:networkSecurityConfig="@xml/network_security_config"
android:roundIcon="@mipmap/ic_launcher_round" android:supportsRtl="true" android:theme="@style/AppTheme">
27:        <receiver android:enabled="true" android:exported="true" android:name="owasp.sat.agoat.ShowDataReceiver"/>
37:        <service android:enabled="true" android:exported="true" android:name="owasp.sat.agoat.DownloadInvoiceService"/>

== Hardcoded keys / secrets / tokens ==
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/R.java:794:        public static final int password = 0x7f070077;
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageTempActivity.java:48:        final EditText password = (EditText) findViewById(R.id.password);
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageTempActivity.java:65:                    sb2.append("password is ");
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageTempActivity.java:66:                    EditText password2 = password;
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageTempActivity.java:67:                    Intrinsics.checkExpressionValueIsNotNull(password2, "password");
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageTempActivity.java:68:                    sb2.append(password2.getText().toString());
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:52:            sQLiteDatabaseOpenOrCreateDatabase.execSQL("CREATE TABLE IF NOT EXISTS users (ID INTEGER NOT NULL PRIMARY KEY
AUTOINCREMENT, username VARCHAR, password VARCHAR)");
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:59:        final EditText password = (EditText) findViewById(R.id.password);
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:64:                sb.append("INSERT INTO users (username, password) VALUES('");
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:69:                EditText password2 = password;
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:70:                Intrinsics.checkExpressionValueIsNotNull(password2, "password");
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:71:                sb.append(password2.getText().toString());
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:46:        final EditText password = (EditText) findViewById(R.id.password);
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:55:            EditText password2 = password;
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:56:            Intrinsics.checkExpressionValueIsNotNull(password2, "password");
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:57:            editor.putString("password", password2.getText().toString());
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/SQLinjectionActivity.java:78:                    strb.append("Username: (" + QryResult.getString(1) + ") password: (" + QryResult.getString(2) + ")\n");
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/ShowDataReceiver.java:18:        Toast.makeText(context, "Username is CrazyUser, Password is CrazyPassword and Key is 123myKey456", 1).show();
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureLoggingActivity.java:46:        final EditText password = (EditText) findViewById(R.id.password);
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureLoggingActivity.java:55:            sb.append("  and Password ");
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureLoggingActivity.java:56:            EditText password2 = password;
/home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureLoggingActivity.java:57:            Intrinsics.checkExpressionValueIsNotNull(password2, "password");
```


I have identified the list of vulnerabilities, which are listed below:

OWASP Mobile Risk	Description	Evidence Found in Decompiled Code
M1	Misuse of Android components or permissions.	allowBackup=true, debuggable=true, exported components (ShowDataReceiver, DownloadInvoiceService).
M2	Sensitive data stored insecurely (SharedPreferences, SQLite, SDCard, temp files).	InsecureStorageSharedPrefs.java, InsecureStorageSQLiteActivity.java, InsecureStorageSDCardActivity.java, InsecureStorageTempActivity.java – passwords stored in plain text.
M3	Unencrypted HTTP communication or improper SSL/TLS.	TrafficActivity.java uses http://demo.testfire.net (HTTP).
M4	Weak or missing authentication mechanisms.	SQLinjectionActivity.java displays passwords directly, lacks proper authentication.
M5	Use of weak or improper cryptography.	javax.crypto.spec.SecretKeySpec with hardcoded keys in ShowDataReceiver.java.
M6	Missing access control or privilege checks.	Exported services without restrictions (DownloadInvoiceService, ShowDataReceiver).
M7	Poor coding practices leading to vulnerabilities (logging, exceptions).	InsecureLoggingActivity.java logs passwords and sensitive info.
M8	App allows tampering or modification.	android:debuggable=true allows easy modification.
M9	Sensitive info (keys, URLs) in code makes reverse engineering easy.	Hardcoded credentials: 'Password is CrazyPassword and Key is 123myKey456'.
M10	Hidden or debug functionality left in production.	Debuggable mode and leftover test URLs (https://github.com/.../AndroGoatInvoice.txt).

Submitted by Nanda Krishnan V *, MSc Cybersecurity*

M1 – Improper Platform Usage

```
1391 /home/kali/Downloads/AndroGoat_jadx/sources/okhttp3/RealCall.java:95:    @Override // okhttp3.Call
1392 /home/kali/Downloads/AndroGoat_jadx/sources/okhttp3/RealCall.java:108:            super("OkHttp %s", RealCall.this.redactedUrl());
1393
1394 == Native libs inside APK ==
1395
1396 == Dev/debug flags & logging ==
1397 /home/kali/Downloads/AndroGoat_apktool_out/AndroidManifest.xml:5:    <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name"
     android:networkSecurityConfig="@xml/network_security_config" android:roundIcon="@mipmap/ic_launcher_round" android:supportsRtl="true" android:theme="@style/AppTheme">
1398 /home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:82:                    Log.d("Error", "Error occurred when inserting data into database: " + e2.getMessage());
1399 /home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/RootDetectionActivity.java:74:            System.out.println(e);
1400 /home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs1Activity.java:84:        System.out.println((Object) ("Score " + String.valueOf(score)));
1401 /home/kali/Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs1Activity.java:85:        System.out.println((Object) ("Level " + String.valueOf(level)));
```

M2 – Insecure Data Storage

```
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:52:            sQLiteDatabaseOpenOrCreateDatabase.execSQL("CREATE TABLE IF NOT EXIS
me VARCHAR, password VARCHAR)");
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:59:        final EditText password = (EditText) findViewById(R.id.password);
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:64:                sb.append("INSERT INTO users (username, password) VALUES('");
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:69:                EditText password2 = password;
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:70:                Intrinsics.checkExpressionValueIsNotNull(password2, "password");
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:71:                sb.append(password2.getText().toString());
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:46:        final EditText password = (EditText) findViewById(R.id.password);
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:55:                EditText password2 = password;
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:56:                Intrinsics.checkExpressionValueIsNotNull(password2, "password");
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:57:                editor.putString("password", password2.getText().toString());
AndroGoat_jadx/sources/owasp/sat/agoat/SQLinjectionActivity.java:78:                    strb.append("Username: (" + QryResult.getString(1) + ") passw
AndroGoat_jadx/sources/owasp/sat/agoat/ShowDataReceiver.java:18:        Toast.makeText(context, "Username is CrazyUser, Password is CrazyPassword and Key is
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureLoggingActivity.java:46:        final EditText password = (EditText) findViewById(R.id.password);
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureLoggingActivity.java:55:                sb.append("  and Password ");
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureLoggingActivity.java:56:                EditText password2 = password;
AndroGoat_jadx/sources/owasp/sat/agoat/InsecureLoggingActivity.java:57:                Intrinsics.checkExpressionValueIsNotNull(password2, "password");
```

M3 – Insecure Communication

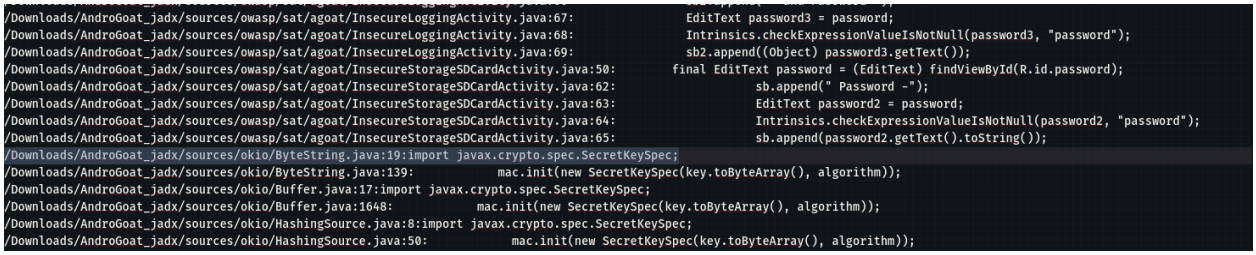
```
/http/HttpHeaders.java:24:    private static final String TOKEN = "([^ \"=]*)";
e.java:12:    private static final String TOKEN = "([a-zA-Z0-9-!#$%&'*+.^_`{|}~]+)";
e.java:47:                String token = parameter.group(2);
e.java:48:                if (token != null) {
e.java:49:                    if (token.startsWith("'") && token.endsWith("'") && token.length() > 2) {
e.java:50:                        charsetParameter = token.substring(1, token.length() - 1);
e.java:52:                        charsetParameter = token;


DownloadInvoiceService.java:64:                Uri url = Uri.parse("https://github.com/satishpatnayak/MyTest/blob/master/AndroGoatInvoice.txt");
TrafficActivity.java:30:    private String httpurl = "http://demo.testfire.net";
TrafficActivity.java:31:    private String httpsurl = "https://owasp.org";
TrafficActivity.java:140:                    Request request = new Request.Builder().url("https://owasp.org").build();
v4/text/util/LinkifyCompat.java:61:            gatherLinks(links, text, PatternsCompat.AUTOLINK_WEB_URL, new String[]{"http://", "https://", "rtsp://"},

v4/content/res/TypedArrayUtils.java:13:    private static final String NAMESPACE = "http://schemas.android.com/apk/res/android";
va:463:            return this.url.startsWith("https://");
atePinner.java:152:                strHost = HttpUrl.parse("http://" + pattern.substring(WILDCARD.length())).host();
atePinner.java:154:                strHost = HttpUrl.parse("http://" + pattern).host();
```

M4 – Insecure Authentication

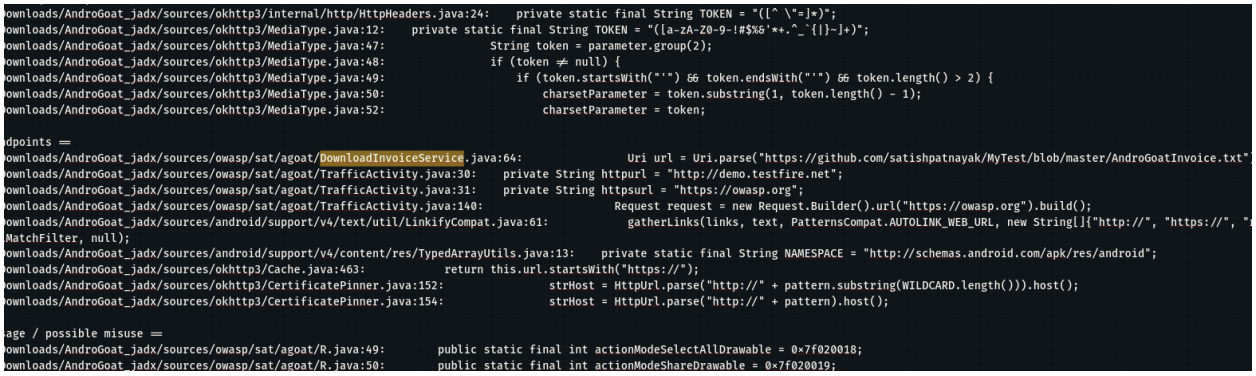
```
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:49:            if (sQLiteDatabaseOpenOrCreateDatabase == null) {
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:52:            sQLiteDatabaseOpenOrCreateDatabase.execSQL("CREATE TABLE IF NOT EXISTS users (ID I
NT, username VARCHAR, password VARCHAR)");
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:75:                    SQLiteDatabase sQLiteDatabase = InsecureStorageSQLiteActivity.this.mDB;
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:76:                    if (sQLiteDatabase == null) {
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSQLiteActivity.java:79:                    sQLiteDatabase.execSQL(qry);
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs1Activity.java:89:        SharedPreferences sharedPreference = getSharedPreferences("score", 0);
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs1Activity.java:99:        SharedPreferences sharedPreferences = getSharedPreferences("score", 0);
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSharedPrefs.java:50:        SharedPreferences sharedPreference = InsecureStorageSharedPrefs.this.getSharedPre
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/SQLinjectionActivity.java:4:import android.database.sqlite.SQLiteDatabase;
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/SQLinjectionActivity.java:17:@Metadata(bv = {1, 0, 3}, d1 =
01e\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0002\n\u0002\u0018\u0002\n\u0000\n\u0002\u0010\u0002\n\u0000\n\u0002\u0018\u0002\n\u0000\u0018\u00002\u00020\u0001B\u0005¢
2\u0010\u0002J\u0012\u0010\u0005\u001a\u00020\u00062\b\u0010\u0007\u001a\u0004\u0018\u00010\bH\u0014R\u0010\u0010\u0003\u001a\u0004\u0018\u00010\u0004X\u0082\u000e¢\u0006\u0002\n\u0
QLinjectionActivity;", "Landroid/support/v7/app/AppCompatActivity;", "()V", "mDB", "Landroid/database/sqlite/SQLiteDatabase;", "onCreate", "", "savedInstanceState", "Landroid/os/Bun

Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/SQLinjectionActivity.java:21:    private SQLiteDatabase mDB;
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/SQLinjectionActivity.java:61:                    SQLiteDatabase sQLiteDatabase = SQLinjectionActivity.this.mDB;
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/SQLinjectionActivity.java:62:                    if (sQLiteDatabase == null) {
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/SQLinjectionActivity.java:65:                    Cursor QryResult = sQLiteDatabase.rawQuery(qry, null);
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSDCardActivity.java:68:                    File externalStorageDirectory = Environment.getExternalStorageDirectory();
Downloads/AndroGoat_jadx/sources/owasp/sat/agoat/InsecureStorageSDCardActivity.java:69:                    Intrinsics.checkExpressionValueIsNotNull(externalStorageDirectory,
 getExternalStorageDirectory()");
```

Submitted by Nanda Krishnan V , MSc Cybersecurity

M5 – Insufficient Cryptography



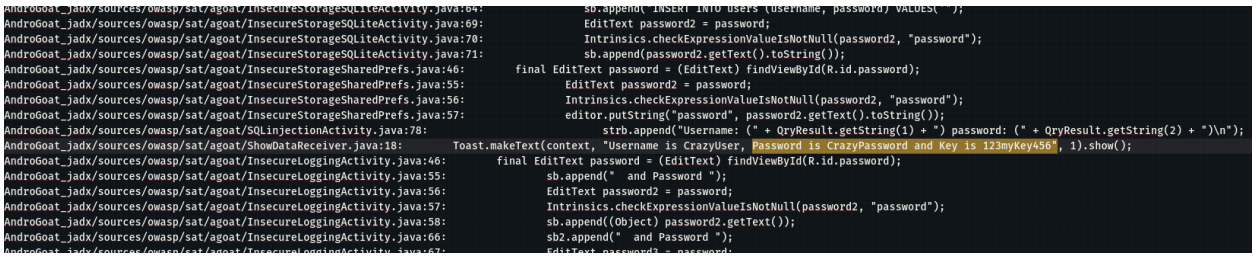
M6 – Insecure Authorization



M7 – Client Code Quality

M8 – Code Tampering

M9 – Reverse Engineering



Submitted by Nanda Krishnan V , MSc Cybersecurity

M10 – Extraneous Functionality

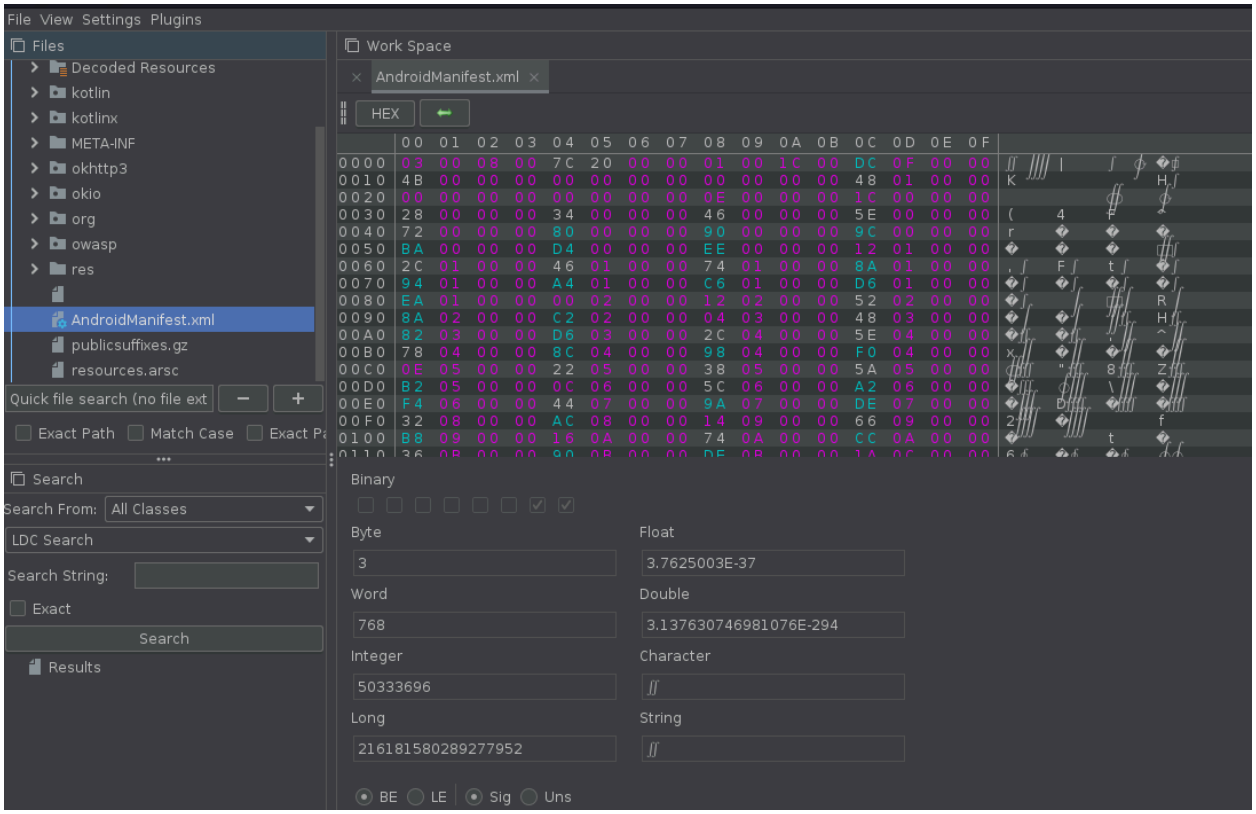
```
        charsetParameter = token.substring(1, token.length() - 1);
        charsetParameter = token;


:64:                Uri url = Uri.parse("https://github.com/satishpatnayak/MyTest/blob/master/AndroGoatInvoice.txt");
  private String httpurl = "http://demo.testfire.net";
  private String httpsurl = "https://owasp.org";
                  Request request = new Request.Builder().url("https://owasp.org").build();
.java:61:             gatherLinks(links, text, PatternsCompat.AUTOLINK_WEB_URL, new String[]{"http://", "https://", "rtsp://

ils.java:13:    private static final String NAMESPACE = "http://schemas.android.com/apk/res/android";
this.url.startsWith("https://");
         strHost = HttpUrl.parse("http://" + pattern.substring(WILDCARD.length())).host();
         strHost = HttpUrl.parse("http://" + pattern).host();
```

Analyzing the file using - ByteCode

Installation of the GUI

```
┌──(kali㉿kali)-[~/Downloads]
└─$ java -jar Bytecode-Viewer-2.13.1.jar

Bytecode Viewer 2.13.1 [Fat Jar] - https://bytecodeviewer.com
Created by @Konloch - https://konloch.com
Presented by https://the.bytecode.club
Cannot set security manager! Are you on Java 18+ and have not enabled support for it?
Because of this, you may be susceptible to some exploits!
Either deal with it or allow it using the -Djava.security.manager=allow parameter.
Extracting Krakatau
Start up took 0 seconds
Successfully extracted Krakatau
Extracting Enjarify
Successfully extracted Enjarify
```

After adding the APK file to the bytecode viewer:



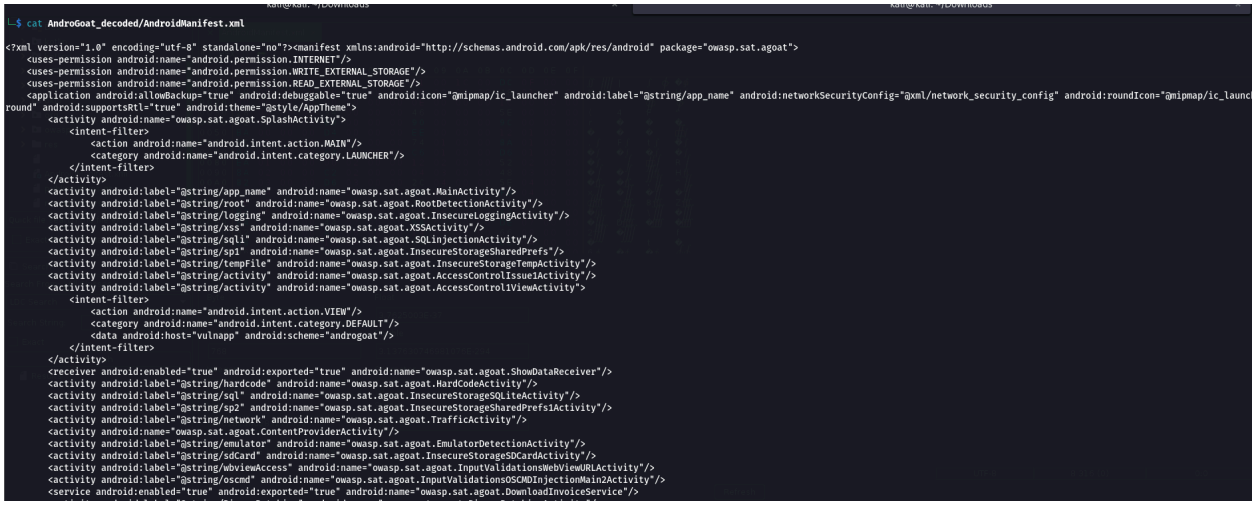
These are not human-readable, so we convert them to a human-readable format using the CLI.

```
┌──(kali㉿kali)-[~/Downloads]
└─$ apktool d AndroGoat.apk -o AndroGoat_decoded

I: Using Apktool 2.7.0-dirty on AndroGoat.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/kali/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...

┌──(kali㉿kali)-[~/Downloads]
└─$
```

Command: apktool d AndroGoat.apk -o AndroGoat_decoded

Action: This is almost identical to the first command you sent, but with the addition of **-o AndroGoat_decoded**.
Explanation: The -o (or --output) flag tells apktool to place all the decoded files (the Smali code, AndroidManifest.xml, resources, etc.) into a **new folder** specifically named AndroGoat_decoded.

We got,



```
└─$ cat AndroGoat_decoded/AndroidManifest.xml

<?xml version="1.0" encoding="utf-8" standalone="no"?><manifest xmlns:android="http://schemas.android.com/apk/res/android" package="owasp.sat.agoat">
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:networkSecurityConfig="@xml/network_security_config" android:roundIcon="@mipmap/ic_launcher_
round" android:supportsRtl="true" android:theme="@style/AppTheme">
        <activity android:name="owasp.sat.agoat.SplashActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <activity android:label="@string/app_name" android:name="owasp.sat.agoat.MainActivity"/>
        <activity android:label="@string/root" android:name="owasp.sat.agoat.RootDetectionActivity"/>
        <activity android:label="@string/logging" android:name="owasp.sat.agoat.InsecureLoggingActivity"/>
        <activity android:label="@string/xss" android:name="owasp.sat.agoat.XSSActivity"/>
        <activity android:label="@string/sqli" android:name="owasp.sat.agoat.SQLinjectionActivity"/>
        <activity android:label="@string/sp1" android:name="owasp.sat.agoat.InsecureStorageSharedPrefs"/>
        <activity android:label="@string/tempFile" android:name="owasp.sat.agoat.InsecureStorageTempActivity"/>
        <activity android:label="@string/activity" android:name="owasp.sat.agoat.AccessControlIssue1Activity"/>
        <activity android:label="@string/activity" android:name="owasp.sat.agoat.AccessControl1ViewActivity">
            <intent-filter>
                <action android:name="android.intent.action.VIEW"/>
                <category android:name="android.intent.category.DEFAULT"/>
                <data android:host="vulnapp" android:scheme="androgoat"/>
            </intent-filter>
        </activity>
        <receiver android:enabled="true" android:exported="true" android:name="owasp.sat.agoat.ShowDataReceiver"/>
        <activity android:label="@string/hardcode" android:name="owasp.sat.agoat.HardCodeActivity"/>
        <activity android:label="@string/sql" android:name="owasp.sat.agoat.InsecureStorageSQLiteActivity"/>
        <activity android:label="@string/sp2" android:name="owasp.sat.agoat.InsecureStorageSharedPrefs1Activity"/>
        <activity android:label="@string/network" android:name="owasp.sat.agoat.TrafficActivity"/>
        <activity android:name="owasp.sat.agoat.ContentProviderActivity"/>
        <activity android:label="@string/emulator" android:name="owasp.sat.agoat.EmulatorDetectionActivity"/>
        <activity android:label="@string/sdCard" android:name="owasp.sat.agoat.InsecureStorageSDCardActivity"/>
        <activity android:label="@string/wbviewAccess" android:name="owasp.sat.agoat.InputValidationsWebViewURLActivity"/>
        <activity android:label="@string/oscmd" android:name="owasp.sat.agoat.InputValidationsOSCMDInjectionMain2Activity"/>
        <service android:enabled="true" android:exported="true" android:name="owasp.sat.agoat.DownloadInvoiceService"/>
```


```
        </activity>
        <activity android:label="@string/app_name" android:name="owasp.sat.agoat.MainActivity"/>
        <activity android:label="@string/root" android:name="owasp.sat.agoat.RootDetectionActivity"/>
        <activity android:label="@string/logging" android:name="owasp.sat.agoat.InsecureLoggingActivity"/>
        <activity android:label="@string/xss" android:name="owasp.sat.agoat.XSSActivity"/>
        <activity android:label="@string/sqli" android:name="owasp.sat.agoat.SQLinjectionActivity"/>
        <activity android:label="@string/sp1" android:name="owasp.sat.agoat.InsecureStorageSharedPrefs"/>
        <activity android:label="@string/tempFile" android:name="owasp.sat.agoat.InsecureStorageTempActivity"/>
        <activity android:label="@string/activity" android:name="owasp.sat.agoat.AccessControlIssue1Activity"/>
        <activity android:label="@string/activity" android:name="owasp.sat.agoat.AccessControl1ViewActivity">
            <intent-filter>
                <action android:name="android.intent.action.VIEW"/>
                <category android:name="android.intent.category.DEFAULT"/>
                <data android:host="vulnapp" android:scheme="androgoat"/>
            </intent-filter>
        </activity>
        <receiver android:enabled="true" android:exported="true" android:name="owasp.sat.agoat.ShowDataReceiver"/>
        <activity android:label="@string/hardcode" android:name="owasp.sat.agoat.HardCodeActivity"/>
        <activity android:label="@string/sql" android:name="owasp.sat.agoat.InsecureStorageSQLiteActivity"/>
        <activity android:label="@string/sp2" android:name="owasp.sat.agoat.InsecureStorageSharedPrefs1Activity"/>
        <activity android:label="@string/network" android:name="owasp.sat.agoat.TrafficActivity"/>
        <activity android:name="owasp.sat.agoat.ContentProviderActivity"/>
        <activity android:label="@string/emulator" android:name="owasp.sat.agoat.EmulatorDetectionActivity"/>
        <activity android:label="@string/sdCard" android:name="owasp.sat.agoat.InsecureStorageSDCardActivity"/>
        <activity android:label="@string/wbviewAccess" android:name="owasp.sat.agoat.InputValidationsWebViewURLActivity"/>
        <activity android:label="@string/oscmd" android:name="owasp.sat.agoat.InputValidationsOSCMDInjectionMain2Activity"/>
        <service android:enabled="true" android:exported="true" android:name="owasp.sat.agoat.DownloadInvoiceService"/>
```


```
        <activity android:label="@string/BinaryPatching" android:name="owasp.sat.agoat.BinaryPatchingActivity"/>
        <activity android:label="@string/clipboard" android:name="owasp.sat.agoat.ClipboardActivity"/>
        <activity android:label="@string/InsecureStorage" android:name="owasp.sat.agoat.InsecureStorageActivity"/>
        <activity android:label="@string/SideChannelLeakage" android:name="owasp.sat.agoat.SideChannelDataLeakageActivity"/>
        <activity android:label="@string/InputValidations" android:name="owasp.sat.agoat.InputValidationsActivity"/>
        <activity android:label="@string/dict" android:name="owasp.sat.agoat.KeyboardCacheActivity"/>
        <meta-data android:name="android.support.VERSION" android:value="26.1.0"/>
        <meta-data android:name="android.arch.lifecycle.VERSION" android:value="27.0.0-SNAPSHOT"/>
    </application>
</manifest>
```

Identified Vulnerabilities mapping OWASP top 10

#	OWASP Mobile Top 10	Evidence from Manifest	Explanation
M1	Improper Platform Usage	Exported Activities and Receivers (ShowDataReceiver, DownloadInvoiceService)	App components are exported, allowing other apps to access them — violates secure component usage.
M2	Insecure Data Storage	InsecureStorageSharedPrefsActivity, InsecureStorageTempActivity, InsecureStorageSQLiteActivity, InsecureStorageSDCardActivity	Sensitive data may be stored unencrypted in shared preferences, temporary files, SQLite databases, or SD cards.
M3	Insecure Communication	TrafficActivity	Potential insecure network communication (HTTP instead of HTTPS or weak SSL validation).
M4	Insecure Authentication	AccessControlIssue1Activity, AccessControlIssue2Activity	Weak or improper authentication checks that can be bypassed.
M5	Insufficient Cryptography	(Likely in smali or code files, not visible in manifest)	Weak or hardcoded encryption algorithms may exist in the code base.
M6	Insecure Authorization	Exported Activities with android:exported="true"	Activities can be invoked externally, leading to privilege escalation or unauthorized access.
M7	Client Code Quality	RootDetectionActivity, EmulatorDetectionActivity	App has root and emulator detection code that may be easily bypassed — poor client-side protection.
M8	Code Tampering	BinaryPatchingActivity	Demonstrates that APK code can be modified or patched — highlights tampering vulnerability.
M9	Reverse Engineering	Readable and descriptive activity names (SQLInjectionActivity, XSSActivity)	Lack of obfuscation makes reverse engineering easier.
M10	Extraneous Functionality	KeyboardCacheActivity, ClipboardActivity	Hidden or debug/test functionalities left in the app may leak sensitive data.

Submitted by Nanda Krishnan V , MSc Cybersecurity